

Method of modification of self-adaptive software systems based on ontology

Dmytro Fedasyuk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
dmytro.v.fedasyuk@lpnu.ua
ORCID: 0000-0003-3552-7454

Illia Lutsyk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
illia.i.lutsyk@lpnu.ua
ORCID: 0000-0001-7117-5332

Abstract— The approaches and methods of implementing self-adaptive software systems have been analyzed. Based on the assessment of the scientific problem, an ontological model of self-adaptive systems has been designed. The constructed model allows to take into account specifics of the subject area during system adaptation. The method of modification of functional characteristics and graphic interface of self-adaptive systems based on ontology is proposed. The developed method allows to generate personalized settings with the possibility of further adapting the system based on the information about user needs, new requirements, and the runtime environment. The use of the ontological model and the adaptation method have been analyzed, and the directions for their improvement have been determined.

Keywords—*ontological model, self-adaptive software, adaptation process, modification of system components.*

I. INTRODUCTION

Software development is a complex process that includes a sequence of specifically defined stages: requirements definition, design, development, testing, implementation, and support. However, the main problem is the modification of functionality after the software implementation process. Adding new features is a resource-intensive process and requires re-development steps considering the new functionality and its integration into the existing system. As a result, there is a problem with effective planning and use of development resources for the integration of new functionality. In addition, adapting an existing system to new requirements requires the involvement of experts from the relevant subject area, who are often not specialists in software engineering [1].

The solution to this specific problem is the development of a software system that allows to dynamically expand the functionality and add resources for the graphical interface. However, most of the existing solutions do not allow modification without the need for static reconfiguration of the whole system.

It should be noted that to provide the ability to dynamically change the characteristics of the created software solution, two types of systems are usually used: adaptive and self-adaptive. The difference in the two approaches lies in the process of determining the need for a process of modifying the characteristics and properties of software solutions. Unlike adaptive systems, self-adaptive systems can independently analyze specific characteristics and subsequently change their behaviour, depending on the obtained assessment of efficiency or productivity [2].

Considering the problems of developing complex software systems, it is important to create methods for implementing self-adaptive software systems by defining the architecture and modification process, which provide for the possibility of dynamically changing the functional characteristics and graphical interface of the software depending on the execution environment and user needs.

II. ANALYSIS OF RECENT RESEARCH AND PUBLICATIONS

Changing and supplementing the requirements specification usually indicates the need to change the functionality, add new functionality, modify the graphical interface by changing the resources and existing graphical elements. One of the key components of adaptive or self-adaptive systems is the designed software architecture based on the use of plugins or additional modules [1]. This solution is effective because it eliminates the high coupling between the additional structures of the program and the main module of the system. However, it should also be noted that defining a plugin architecture requires the creation of additional structures and programming interfaces that provide a clean mechanism for attaching new modules or services.

General principles and methods of design and implementation of self-adaptive systems are defined in number of studies. In particular, in [3, 4], the authors described the problems of designing these systems, namely, the problems of determining new unified processes of software adaptation and decentralization of system elements. As a result, new directions for the study of self-adaptive systems and possible difficulties associated with the development of complex software are presented. A detailed review of solutions for creating self-adaptive systems and their taxonomy is presented in the study [5]. Based on the taxonomy of approaches, authors identified new directions for the development of the process of software systems adaptation, in particular, the use of contextual adaptation.

Studies on the creation of self-adaptive systems based on feedback and control diagrams are presented in [6, 7]. In [6], the researchers note that the use of feedback diagrams (feedback-loop) is the basis for dynamic control of the behaviour of technical systems in various industries. As a result, the authors created a reference model based on a feedback loop, which provided the ability to save over time and dynamically manage system settings. Despite the overall effectiveness of the proposed approach, the system requires an additional definition of a domain-specific language to fully define the objects of the subject area. It should also be noted that the proposed solution is not universal, but a

reference, since the work defines the system in terms of a specific field of application.

Research of the process of design and development of controlled systems based on reference models is reflected in [7]. The authors presented a taxonomy of change management strategies that allow to analyze their applicability to functional and non-functional properties. However, not all principles and methods presented in the paper can be directly applied to software adaptation problems without the need to design new patterns or data structures.

Another problem that arises when creating adaptive and self-adaptive systems is the definition of software requirements. The design of these systems requires a more thorough processing of the subject area, since the better the objects of the subject area are described, the more correctly the program will dynamically react to changing requirements. However, traditional approaches to the design and definition of the requirements of self-adaptive systems do not provide a sufficiently complete and clear presentation of the concepts of the subject area. The solution to the problem is the use of a domain ontology, which will allow the reuse of concepts and classes of the domain to form new concepts and relationships in accordance with changing requirements.

The study of the use of the ontological approach in the creation of self-adaptive systems is revealed in the work of Pecanha, Duarte, Falbo, Souza [8]. In their work, the authors presented a comprehensive ontological model for requirements-based self-adaptive systems. As a result of the study, the authors conducted an experimental test and verification of the designed model based on five competency questions. Despite a fairly complete description of the proposed ontological model of the domain, it can only be used to support and re-engineer adaptive software systems based on requirements.

The use of the conceptual model in self-adaptive systems to implement the management process with predictive models is presented in the study Angelopoulos, Papadopoulos, Souza [9]. Using the example of a meeting scheduling system, the researchers uncovered the process of adapting software systems based on a dynamic relationship between requirements and possible adaptations. This process is based on the use of a conceptual domain model in combination with specific indicators to determine what changes need to be introduced into the system. However, despite the successful process of adaptation of a certain system, the authors note that for a better assessment of the effectiveness, the implementation of adaptation for various subject areas should be considered.

The analysis of research on approaches to the creation and maintenance of self-adaptive systems confirmed the relevance of creating and improving methods of adapting software systems. Among the existing methods, it is typical to use the conceptual model of the domain, which allows to determine the relationships and correlation between new requirements for software and the necessary actions to adapt functional and non-functional features. The main problem during such an adaptation process is the static structure of the corresponding model, which, in turn, does not allow making changes to the architecture or considering new devices on which the corresponding software will be installed.

Accordingly, it is advisable to use generalized ontological models that make it possible to unify the adaptation process for software systems with different subject areas.

Thus, **the purpose of the study** is to create a new method of the components modification of self-adaptive software systems based on ontological models that will universalize the process of dynamic adaptation of functional characteristics and graphical user interface in accordance with changing software requirements.

III. THE PROCESS OF ADAPTATION OF SOFTWARE SYSTEMS BASED ON ONTOLOGIES

A. Ontological model of self-adaptive software system

The use of conceptual models in the design of self-adaptive systems, as noted earlier, not only allows to determine the structural organization but also provides a representation of the relationship between the objects of a particular subject area. In addition, this model can be used not only in the initial design of the system, but in reengineering and reconfiguration of software.

In its turn, the use of classical approaches to the implementation of software systems based on models usually does not allow to correctly and effectively respond to the changes and is rather resource-intensive. In addition, some models do not allow the reconfiguration without reassembling and deploying the system. One of the effective solutions to these problems is the use of the ontological approach [10, 11].

Formalized ontological model can be represented as a set of concepts, relationships and attributes of the subject area. Based on previous research [12], we have expanded the ontological model, which allows to consider the features and characteristics of different subject areas. In addition, the updated model does not create a high coupling in the structure of the functionality of the software system. The formed top-level ontology for self-adaptive systems can be defined as a combination of 5 sets:

$$O_{sys} = \langle C_{sys}, R_{sys}, RI_{sys}, Ind_{sys}, Prop_{sys} \rangle, \quad (1)$$

where C_{sys} – a set of concepts that define the main objects of self-adaptive software systems;

R_{sys} – a set of connections and relationships between the concepts of the ontological model;

RI_{sys} – a set of semantic rules that allow to make logical conclusions based on information about the object and relationships with other elements;

Ind_{sys} – a set of individuals of concepts of the ontological model of self-adaptive systems;

$Prop_{sys}$ – a set of properties that complement / expand information about the objects of self-adaptive systems.

In order to obtain correct results during adaptation, four sets are used: a set of concepts, a set of attributes, a set of rules, and a set of relationships between concepts. The formed set of connections and relations between the objects of the self-adaptive system allows to dynamically identify and respond to changes in software requirements or new user needs and contains the following elements:

1. «has» – the relationship is established between the base class and the subclass. For example, between the classes "Functional component" and "Module";
2. «uses» – the relationship between the entities "User" and "Software System" allows to determine which version of the software system the user is using;
3. «contains» – to implement the possibility of selecting several components, this relationship is established between the entities "Software system" and "Software component";
4. «has_impact_on» – connection between the "Requirement" and "Software Component" objects, to be able to identify the impact of new functional and non-functional requirements;
5. «has_requirements» – the relationship between the "User" and "Requirement" entities;
6. «uses_UI_elements» – the established connection between the entities "Functional component" and "UI component" allows to identify the relation between functional features and graphical elements.

As a result, the obtained ontological model for self-adaptive systems, which allows to consider new changes in requirements and to carry out dynamic modification of the system, is shown in Fig. 1.

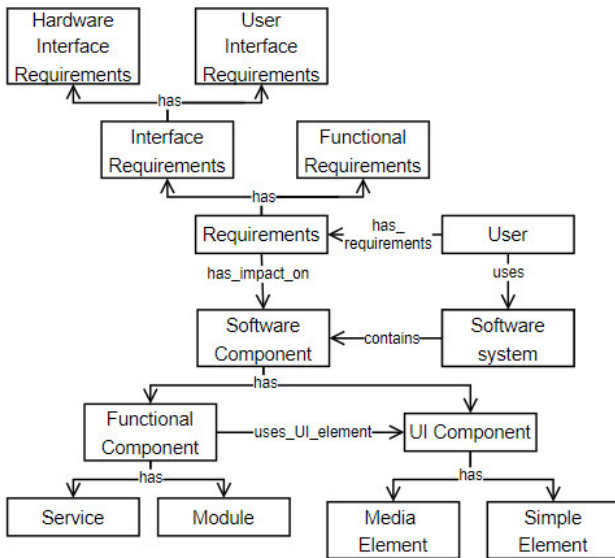


Fig. 1. Ontological model of self-adaptive software system

B. Method of modification of self-adaptive software systems based on ontology

The proposed ontological model of the domain for self-adaptive systems allows to determine the necessary actions to respond to changes in the requirements or needs of the user. However, this model is only the part of the process of generating software settings. The conceptual scheme of the process of adaptation of system characteristics, taking into account the elements of the upper level of interaction, is presented in Fig. 2.

The main external objects for the process are "User" and "Active Device". In fact, to initiate the process of changing the settings, the user sends the selected information, which serves as an identifier for the necessary changes in the behaviour or appearance of the system. During data

processing, the system makes an inverse request for information about the active device, which will help identify the possibility of changes in specific characteristics. As a result, the generated settings are considered during the process of system components modification and the user is presented with the adapted characteristics.

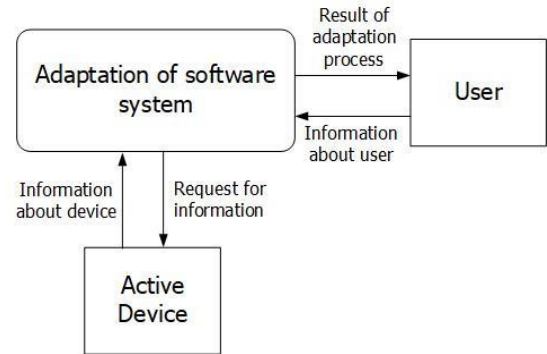


Fig. 2. Conceptual scheme of modification of self-adaptive software system

The detailed process of components modification of self-adaptive systems is presented in Fig. 3.

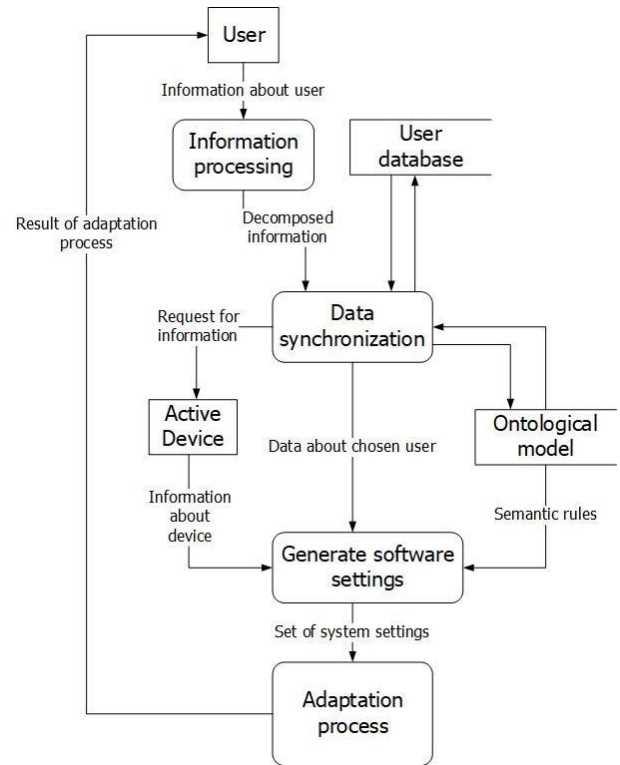


Fig. 3. Data flow diagram of the process of modification of a self-adaptive software system

Based on the specified process, the method of changing the system components consists of the following steps:

1. Processing of information received from the user (user identification, verification of the correctness of the information);
2. Data synchronization, which allows to create or modify the necessary records both in the database and in the ontological knowledge base;
3. Generating settings - the stage processes data about the selected user and forms the necessary system

settings, based on the connections and properties of ontology concepts;

4. Adaptation of the software system includes modification of functional characteristics and graphical interface based on a set of formed settings.

According to the depicted process, the ontological model is used in two key stages: data synchronization and generation of software settings. The first stage allows to store selected information about the user in the ontological knowledge base in order to facilitate further requests to modify the characteristics of the system. The second stage actually launches the semantic reasoner, which begins the elaboration of semantic rules defined in the ontological model. These two stages are interrelated because the result of data synchronization between the database and the knowledge base is responsible for the correct filling of the knowledge base only with updated information.

IV. DISCUSSION AND PROSPECTS FOR FUTURE RESEARCH

The proposed ontological model of self-adaptive software systems and the method of dynamic modification of functional characteristics and graphical interface allow to consider the peculiarities of the use of software for an individual user. It should be noted that the improved conceptual model of the system can be used regardless of the subject area. This feature is explained by the fact that the main structural dependencies are only between the requirements and the corresponding software components. In its turn, this weak coupling will help to improve the time to optimize the system performance during adaptation.

Using an improved ontological model, a method for modifying self-adaptive software systems has been developed. The new approach does not require the creation of additional structural elements while adding new functionality to the system. In addition, the method takes into account information about the active device, which in combination with information about software requirements will improve the result of the modification.

Analyzing the obtained ontological model and method of modification of system components, it should be noted that in order to ensure the universality of the adaptation process it is necessary to improve the method of processing semantic rules in further research. Current solutions allow to process only all available instances of entities of the ontological model. Considering different number of concepts and relationships, the processing time for all rules can increase. In addition, further research is planned to improve the mechanism for changing the functionality, which will allow the adaptation of the software for different platforms.

V. CONCLUSIONS

The problems of designing and implementing self-adaptive software systems have been analyzed. The conducted analysis of scientific research confirms the relevance of creating new methods and tools for software adaptation. It is also established that the available methods of implementation of self-adaptive systems are not universal, as they depend on the characteristics of a particular subject area. The solution to this problem is the use of conceptual and ontological models that provide the ability to make

changes without the need for recompilation and redeployment of the system.

An ontological model of self-adaptive systems has been designed, which allows to consider the dependencies between the new software requirements and the system components to which the requirements apply. The defined relationships and concepts of the ontological model make it possible to dynamically define new software configurations by using the semantic reasoner for ontology rules processing.

A method of modifying the functional components and properties of the graphical interface for self-adaptive systems has been proposed, which provides the ability to create new settings for software in accordance with new requirements or user needs. Compared to the available approaches, the use of the designed method of adaptation will allow to avoid high coupling between system modules and the objects of the subject area.

REFERENCES

- [1] S. Malek, K. Canavera and N. Esfahani "Automated inference techniques to assist with the construction of self-adaptive software", in "Managing trade-offs in adaptable software architectures". Cambridge, MA: Morgan Kaufmann, 2017, pp. 131-154, doi: 10.1016/b978-0-12-802855-1.00006-x.
- [2] L. Sabatucci, V. Seidita and M. Cossentino, "The Four Types of Self-adaptive Systems: A Metamodel", *Intelligent Interactive Multimedia Systems and Services* 2017, pp. 440-450, 2017. doi: 10.1007/978-3-319-59480-4_44.
- [3] R. de Lemos et al., "Software engineering for self-adaptive systems: A second research roadmap", in *Software Engineering for Self-Adaptive Systems II*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 1-32, doi: 10.1007/978-3-642-35813-5_1.
- [4] C. Krupitzer, F. Roth, S. VanSyckel, G. Schiele and C. Becker, "A survey on engineering approaches for self-adaptive systems", *Pervasive and Mobile Computing*, vol. 17, pp. 184-206, 2015. doi: 10.1016/j.pmcj.2014.09.009.
- [5] F. Macias-Escrivá, R. Haber, R. del Toro and V. Hernandez, "Self-adaptive systems: A survey of current approaches, research challenges and applications", *Expert Systems with Applications*, vol. 40, no. 18, pp. 7267-7279, 2013. doi: 10.1016/j.eswa.2013.07.033.
- [6] N. M. Villegas Machado, H. A. Müller and G. T. Morimitsu, "On designing self-adaptive software systems", *Sistemas y Telemática*, vol. 9, no. 18, 2011, pp. 29 – 51. doi: 10.18046/syt.v9i18.1076.
- [7] A. Filieri, et al. "Control strategies for self-adaptive software systems" *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, vol. 11, no. 4, 2017, pp. 1-31.
- [8] C. Peçanha, B. Duarte and V. Souza, "RASO: An ontology on requirements for the development of adaptive systems", in *Workshop Em Engenharia De Requisitos 2018*, 2018.
- [9] K. Angelopoulos, A. V. Papadopoulos, V. E. S. Souza and J. Mylopoulos "Engineering self-adaptive software systems", *ACM Transactions on Autonomous and Adaptive Systems*, vol. 13 no. 1, 2018, pp. 1-27. doi: 10.1145/3105748.
- [10] M. Dyvak, A. Kovbasisty, and A. Melnyk "Recognition of Relevance of Web Resource Content Based on Analysis of Semantic Components," in *Advanced Computer Information Technologies (ACIT 2019): Conference proceedings*, 2019, pp. 297-302, doi: 10.1109/ACITT.2019.8779897.
- [11] V. Lytvyn, Y. Burov, V. Vysotska, and V. Hryhorovych "Knowledge Novelty Assessment During the Automatic Development of Ontologies," in *IEEE Third International Conference on Data Stream Mining & Processing (DSMP)*, 2020, pp. 372-377, doi: 10.1109/DSMP47368.2020.9204124.
- [12] D. Fedasyuk and I. Lutsyk, "Tools for adaptation of a mobile application to the needs of users with cognitive impairments," in *2021 IEEE 16th International Conference on Computer Sciences and Information Technologies (CSIT)*, 2021, pp. 321-324, doi: 10.1109/CSIT52700.2021.9648702.